

فاز ۹: امنیت (Security) – کامل و عمیق

امنیت یه لایه نیست، یه اصل هست که باید توی تمام لایه‌های برنامه (از کد تا سرور) رعایت بشه. ما اینجا روی مفاهیمی تمرکز می‌کنیم که یه بک‌اند کار جاوا (مخصوصاً با Spring Security) باید بلد باشه.

9.1 مفاهیم پایه و حیاتی امنیت

؟ سوال مصاحبه‌ای ۱: Authentication و Authorization چه فرقی دارن؟ (سوال تکراری و

پایه‌ای)

جواب:

• **Authentication (احراز هویت):** پاسخ به سوال "تو کی هستی؟". فرآیند تأیید هویت کاربر (مثلاً با بررسی نام کاربری و رمز عبور). اگر هویت تأیید نشه، خطای 401 Unauthorized برگردونده میشه.

• **Authorization (مجوزدهی/کنترل دسترسی):** پاسخ به سوال "تو چه کاری می‌تونی انجام بدی؟". بعد از احراز هویت، مشخص می‌کنیم که این کاربر به چه منابعی (APIها، صفحات) دسترسی داره. مثلاً کاربر عادی نمیتونه پنل ادمین رو ببینه. اگر دسترسی نداشته باشه، خطای 403 Forbidden برگردونده میشه.

مثال ساده:

• پاسپورت تو = Authentication (ثابت می‌کنه تو شهروندی).

• ویزای تو = Authorization (ثابت می‌کنه اجازه داری وارد یه کشور خاص بشی).

؟ سوال مصاحبه‌ای ۲: چرا نباید پسورد رو به صورت Plain Text ذخیره کنیم؟ راه درستش

چیه؟

جواب: اگر دیتابیس به هر دلیلی (هک، خطای انسانی) لو بره، پسورد تمام کاربران فاش میشه. چون خیلی از کاربران از یه پسورد برای سایت‌های مختلف استفاده می‌کنن، این یه فاجعه‌ست.

راه حل: Hashing (درهم‌سازی)

• هش کردن: یه الگوریتم ریاضی یک‌طرفه (One-way) است. یعنی از روی هش، هرگز نمیشه به پسورد اصلی رسید.

• Salt (نمک): یه رشته تصادفی یکتا که به هر پسورد اضافه میشه و بعد از هش گرفته میشه. این کار حمله با جدول‌های رنگین‌کمان (Rainbow Table Attacks) رو خنثی می‌کنه.

• الگوریتم‌های مناسب: BCrypt، SCrypt یا Argon2. این الگوریتم‌ها عمداً کند (Slow) طراحی شدن تا حملات Brute-Force رو سخت کنن.

• در عمل (جاوا/Spring Security): از BCryptPasswordEncoder استفاده می‌کنیم.

9.2 احراز هویت مدرن: JWT (JSON Web Token)

؟ سوال مصاحبه‌ای ۳: JWT چیه، چه ساختاری داره و چطور کار می‌کنه؟ (سوال طلایی)

مصاحبه‌های Spring

جواب: JWT یه استاندارد باز (RFC 7519) برای انتقال امن اطلاعات (Claims) بین دو طرف به صورت یه JSON فشرده و امضا شده است. بزرگترین مزیتش Stateless (بدون وضعیت) بودنش هست، یعنی سرور نیازی به ذخیره Session ندارد و مقیاس‌پذیری رو بالا می‌بره.

ساختار یه JWT از سه بخش Base64-encoded تشکیل شده که با نقطه جدا

میشن: HEADER.PAYLOAD.SIGNATURE

1.Header: نوع توکن (JWT) و الگوریتم امضا (مثلاً HS256).

```
1 { "alg": "HS256", "typ": "JWT" }
```

```
2 { "sub": "123", "name": "Ali", "role": "ADMIN", "exp": 1680000000 }
```

Signature (امضا): مهم‌ترین بخش برای امنیت. با این فرمول ساخته میشه

```
1 HMACSHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), SECRET_KEY )
```

این امضا تضمین می‌کنه که توکن در مسیر بین کلاینت و سرور دستکاری (Tamper) نشده.

چون اگه Payload تغییر کنه، امضا دیگر معتبر نخواهد بود

فرآیند کار

• کاربر با username/password لاگین می‌کنه.

• سرور هویت رو چک می‌کنه و یک JWT امضاء شده برمی‌گردونه.

• کلاینت JWT رو ذخیره می‌کنه (مثلاً توی Local Storage یا Cookie).

• در هر درخواست بعدی، کلاینت توکن رو در هدر Authorization می‌فرسته:

```
1 Authorization: Bearer <token>
```

• سرور بدون مراجعه به دیتابیس، با استفاده از SECRET_KEY امضای توکن رو چک می‌کنه. اگر معتبر بود،

اطلاعات کاربر رو از Payload استخراج می‌کنه و پردازش انجام میشه.

نکته حیاتی: Payload فقط Base64 Encode شده، نه Encrypt! بنابراین هرگز اطلاعات

حساس مثل پسورد رو توی JWT قرار نده. هر کسی که توکن رو داشته باشه، می‌تونه

Payload رو بخونه

9.3 آسیب‌پذیری‌های رایج (OWASP Top 10)

? سوال مصاحبه‌ای ۴: SQL Injection چیست و چگونه در جاوا از آن جلوگیری می‌کنی؟

جواب: وقتی ورودی کاربر مستقیماً (مثلاً از طریق concatenation رشته‌ها) به یک کوئری SQL اضافه بشه، و مهاجم بتونه با تزریق کد SQL ساختار کوئری رو تغییر بده.

```
1 -- a.
2 -- username: ' OR '1'='1' ورودی کاربر در فیلد
3 -- :کوئری نهایی
4 SELECT * FROM users WHERE username = ' OR '1'='1' AND password = '...'
```

راه حل در جاوا

• هرگز و مطلقاً هرگز با + زدن یا String.format() کوئری SQL نساز!

• همیشه از Parameterized Queries (Prepared Statements) استفاده کن.

• در JDBC: استفاده از PreparedStatement

```
1 String query = "SELECT * FROM users WHERE username = ?";
2 PreparedStatement ps = connection.prepareStatement(query);
3 ps.setString(1, userInput); // اینجا ورودی امن می‌شود
```

? سوال مصاحبه‌ای ۵: XSS (Cross-Site Scripting) و CSRF رو خیلی خلاصه بگو و راه

حلشون چیه؟

• XSS (حمله با اسکریپت بین سایتی): مهاجم یه اسکریپت مخرب جاوااسکریپت رو توی وبسایت قرار میده که در مرورگر قربانی اجرا میشه.

• راه حل: مهم‌ترین کار همیشه Escape/Encode کردن خروجی‌ها قبل از نمایش در HTML هست.

کتابخانه‌هایی مثل Thymeleaf این کار رو خودکار انجام میدن.

• CSRF (جعل درخواست بین سایتی): مهاجم قربانی رو فریب میده تا وقتی توی سایت A لاگین هست، ناخواسته یک درخواست مخرب (مثلاً انتقال پول) به سایت A بفرسته.

• راه حل: استفاده از CSRF Token (یه مقدار تصادفی مخفی در فرم‌ها که توسط سرور تولید و بررسی

میشه). نکته برای بک‌اند کار: اگر API تو Stateless هست و با JWT کار می‌کنی (توکن در هدر میاد، نه

Cookie)، به طور پیش‌فرض در برابر CSRF ایمن هستی و معمولاً CSRF protection رو غیرفعال می‌کنن.

اما برای برنامه‌های سنتی که با Session و Cookie کار می‌کنن، حتماً باید فعال باشه.

9.4 جمع‌بندی بهترین روش‌های امنیتی (Security Best Practices)

• اصل حداقل دسترسی (Principle of Least Privilege): به هر کاربر یا سرویس فقط و فقط کمترین سطح

دسترسی که برای انجام کارش نیاز داره رو بده.

• لاگ‌گیری و مانیتورینگ: تمام تلاش‌های ناموفق برای ورود، خطاهای ۴۰۱ و ۴۰۳، و فعالیت‌های مشکوک رو لاگ کن.

• به‌روزرسانی وابستگی‌ها: همیشه کتابخانه‌ها رو آپدیت نگه دار. آسیب‌پذیری‌های معروف (CVEs) دائماً کشف میشن.

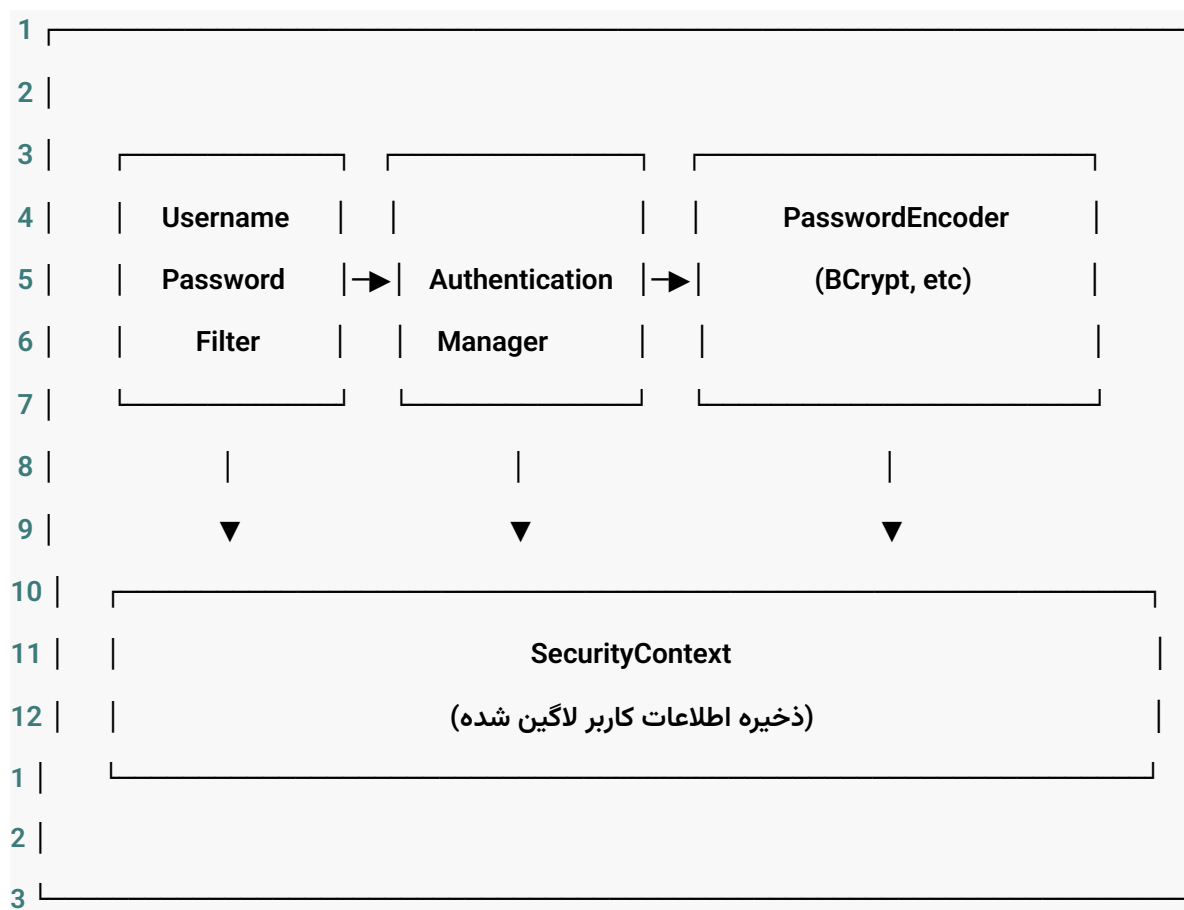
• HTTPS همه جا: هرگز، تأکید می‌کنم هرگز از HTTP ساده در Production استفاده نکن. همیشه SSL/TLS رو فعال کن.

۲. Spring Security

۹.۵ Spring Security (امنیت در Spring Boot)

Spring Security فریم‌ورک اصلی امنیت در اکوسیستم Spring هست.

۹.۵.۱ مفاهیم اصلی Spring Security



۹.۵.۲ PasswordEncoder (رمزنگاری پسورد)

```

1  @Configuration
2  class SecurityConfig {
3
4      @Bean
5      public PasswordEncoder passwordEncoder() {
6          return new BCryptPasswordEncoder(); // قوی و استاندارد
7          //یا: return new SCryptPasswordEncoder();
8          // یا: return new Argon2PasswordEncoder();
9      }
10 }
11 // استفاده
12 @Service
13 class UserService {
14     @Autowired
15     private PasswordEncoder passwordEncoder;
16
17     public void registerUser(String username, String rawPassword) {
18         String encodedPassword = passwordEncoder.encode(rawPassword);
19         // در دیتابیس encodedPassword ذخیره
20     }
21
22     public boolean checkPassword(String rawPassword, String encodedPassword)
23     {
24         return passwordEncoder.matches(rawPassword, encodedPassword);
25     }
26 }

```

۹.۵.۳ UserDetailsService (بارگذاری کاربر)

```
1 @Service
2 class CustomUserDetailsService implements UserDetailsService {
3
4     @Autowired
5     private UserRepository repository;
6
7     @Override
8     public UserDetails loadUserByUsername(String username)
9         throws UsernameNotFoundException {
10
11         User user = repository.findByUsername(username)
12             .orElseThrow(() -> new UsernameNotFoundException("User not found"));
13
14         return User.builder()
15             .username(user.getUsername())
16             .password(user.getPassword())
17             .authorities(user.getRoles().stream()
18                 .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
19                 .toArray(String[]::new))
20             .build();
21     }
22 }
```



```
1 @Component
2 class JwtService {
3
4     @Value("${jwt.secret}")
5     private String secretKey;
6
7     public String generateToken(String username, List<String> roles) {
8         return Jwts.builder()
9             .setSubject(username)
10            .claim("roles", roles)
11            .setIssuedAt(new Date())
12            .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 24h
13            .signWith(SignatureAlgorithm.HS256, secretKey)
14            .compact();
15     }
16
17     public String extractUsername(String token) {
18         return Jwts.parser()
19             .setSigningKey(secretKey)
20             .parseClaimsJws(token)
21             .getBody()
22             .getSubject();
23     }
24
25     public boolean isTokenValid(String token) {
```

```
7     try {
8         Jwts.parser().setSigningKey(secretKey).parseClaimsJws(token);
9         return true;
10    } catch (JwtException e) {
11        return false;
12    }
13 }
14 }
15
16 // JWT Filter
17 @Component
18 class JwtAuthenticationFilter extends OncePerRequestFilter {
19
20     @Autowired
21     private JwtService jwtService;
22
23     @Autowired
24     private CustomUserDetailsService userDetailsService;
25
26     @Override
27     protected void doFilterInternal(HttpServletRequest request,
1         HttpServletResponse response,
2         FilterChain filterChain) {
3         String authHeader = request.getHeader("Authorization");
4
5         if (authHeader != null && authHeader.startsWith("Bearer ")) {
```

```

6     String token = authHeader.substring(7);
7     String username = jwtService.extractUsername(token);
8
9     if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
10         UserDetails userDetails = userDetailsService.loadUserByUsername(username);
11         SecurityContextHolder.getContext().setAuthentication(
12             new UsernamePasswordAuthenticationToken(userDetails, null, userDetails.getAuthorities())
13         );
14     }
15 }
16 filterChain.doFilter(request, response);
17 }
18 }

```

۹.۵.۵ تنظیمات امنیتی (Security Filter Chain)

```

1 @Configuration
2 @EnableWebSecurity
3 @EnableMethodSecurity
4 class SecurityConfig {
5
6     @Bean
7     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
8         return http
9             // ۱. CSRF (غیرفعال کن برای JWT)
10             .csrf(csrf -> csrf.disable())
11         ;
12     }
13 }

```

```

6      // تنظیم دسترسی ها ۲.
7      .authorizeHttpRequests(auth -> auth
8          .requestMatchers("/api/auth/**").permitAll() // لاگین اثبات نام
9          .requestMatchers("/api/admin/**").hasRole("ADMIN")
10         .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
11         .anyRequest().authenticated()
12     )
13
14     // Session مدیریت (JWT Stateless برای)
15     .sessionManagement(session -> session
16         .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
17     )
18
19     // JWT Filter اضافه کردن ۴.
20     .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
21
22     .build();
23 }
24 }

```

۹.۵.۶ استفاده از @PreAuthorize و @Secured

```

1 @RestController
2 @RequestMapping("/api/users")
3 class UserController {
4
5     // می‌تونه ADMIN فقط

```

```

6  @PreAuthorize("hasRole('ADMIN')")
7  @GetMapping("/all")
8  public List<User> getAllUsers() { ... }
9
10 // USER یا ADMIN
11 @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
12 @GetMapping("/{id}")
13 public User getUser(@PathVariable Long id) { ... }
14
15 // بررسی اینکه کاربر خودش باشد
16 @PreAuthorize("#id == authentication.principal.id")
17 @PutMapping("/{id}")
18 public User updateUser(@PathVariable Long id, @RequestBody User user) { ... }
19
20 // فقط USER
21 @Secured("ROLE_USER")
22 @PostMapping("/profile")
23 public User updateProfile() { ... }
24 }

```

۹.۵.۷ Method Security (در Service)

```

1 @Service
2 class OrderService {
3
4  // فقط ADMIN
5  @PreAuthorize("hasRole('ADMIN')")

```

```

6 public List<Order> getAllOrders() { ... }
7
8 // مدیر شرکت یا صاحب سفارش
9 @PreAuthorize("hasRole('ADMIN') or #userId == authentication.principal.id")
10 public Order getOrder(Long userId, Long orderId) { ... }
11 }

```

۹.۵.۸ Authentication Entry Point (مدیریت خطاهای امنیتی)

```

1 @Component
2 class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint {
3
4     @Override
5     public void commence(HttpServletRequest request,
6                           HttpServletResponse response,
7                           AuthenticationException authException) {
8         response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
9         response.setContentType(MediaType.APPLICATION_JSON_VALUE);
10        response.getWriter().write("""
11            {
12                "error": "Unauthorized",
13                "message": "Please login first"
14            }
15        """);
16    }
17 }

```